Evaluating Operator Fusion and the Memory Wall for GNNs on Edge NPUs

Yusuf Talha ARABACI
Dept. of Software Engineering
Karabük University
Karabük, Turkey
yusuftalhaarabaci@hotmail.com

Emrullah DEMİRAL
Dept. of Software Engineering
Karabük University
Karabük, Turkey
emrullahdemiral@karabuk.edu.tr

Ömer Faruk ACAR
Dept. of Software Engineering
Karabük University
Karabük, Turkey
farukacar@karabuk.edu.tr

***Abstract*—The integration of Neural Processing Units (NPUs) into consumer processors, such as the Intel Core Ultra (Meteor Lake) architecture, has enabled high-frequency AI inference on edge devices. While these accelerators are efficient for dense, regular workloads, Graph Neural Networks (GNNs) present unique challenges due to sparse computation and irregular memory access patterns. This paper provides a cross-architectural evaluation of GNNs on the Intel Core Ultra NPU, contrasting its performance with integrated GPU (iGPU) and CPU counterparts. We introduce an analytical framework based on two metrics: the Fusion Gain Ratio (FGR) and the Compilation Efficiency Index (CEI), to characterize the optimization returns of NPU compilers. Our empirical results, encompassing 10 distinct models across 3 hardware backends, show that while the NPU achieves competitive throughput for simplified GNNs, it faces a "Fusion Overhead Paradox" where aggressive operator fusion leads to performance regression in shallow graphs. We identify a significant "Memory Wall" bottleneck, where the NPU's performance is limited by its memory subsystem rather than its peak arithmetic capability. We conclude by exposing hardware-software maturity gaps, where operator fallbacks in dynamic models currently bottleneck edge intelligence.**

***Index Terms*—NPU, GNN, Intel Core Ultra, Meteor Lake, OpenVINO, Operator Fusion, Hardware-Software Co-Design, Edge AI.**

I. Introduction

Neural Processing Units (NPUs) represent a paradigm shift in edge computing, particularly within the Intel Core Ultra (Meteor Lake) ecosystem. While traditional accelerators are optimized for the spatial locality and dense matrix operations of CNNs, Graph Neural Networks (GNNs) introduce irregular dataflows that challenge these hardware assumptions. Our investigation exposes a stark reality: optimizations that yield $10\times$ gains in vision tasks can trigger severe performance regressions in graph structures. Specifically, we report an 18-fold latency penalty for quantized GCN models on the NPU—a finding that contradicts the fundamental promise of post-training quantization.

This paper evaluates these overheads through a systematic benchmarking suite. Beyond raw latency, we propose a diagnostic framework centered on two metrics: the Fusion Gain Ratio (FGR) and the Compilation Efficiency Index (CEI). Our results confirm that GNN execution on consumer NPUs is primarily constrained by a "Memory Wall," where system-wide DRAM latency dominates over peak arithmetic capacity. Furthermore, we identify a "Fusion Overhead Paradox," where aggressive operator merging by the compiler bottlenecks rather than accelerates sparse graph intelligence.

II. Background and Related Work

The efficacy of graph intelligence at the edge is fundamentally constrained by the architectural gap between GNN dataflows and general-purpose accelerators. Unlike CNNs, which benefit from the high spatial locality and structured data reuse of dense matrix engines, GNNs rely on sparse-dense matrix multiplications (SpMM) that trigger irregular, non-sequential memory access patterns [1]. While custom ASIC designs such as HyGCN [2] and GRIP [3] mitigate these issues through hardware-native sparse engines and specialized on-chip scratchpad management, consumer NPUs like the Intel AI Boost are primarily optimized for the streaming data flows characteristic of computer vision workloads [4].

This optimization for regular grids creates a significant "Memory Wall" for graph workloads, where the NPU's internal vector engines are frequently stalled while waiting for vertex features from system DRAM [5]. Existing literature suggests that while integrated GPUs (iGPUs) masked this latency through massive parallelism and high memory bandwidth, the power-constrained nature of NPUs necessitates aggressive graph-level optimizations such as operator fusion to minimize off-chip traffic [6]. However, as GNNs move toward dynamic attention mechanisms [7], the complexity of mapping these irregular subgraphs to static NPU primitives becomes a significant toolchain bottleneck. Our work evaluates whether modern AI compilers can bridge this gap without incurring the "Fusion Penalties" observed in recent pass-level profiling studies [8].

III. Methodology

A. Experimental Setup

Experiments were conducted on a laptop environment (Huawei MateBook) featuring an Intel Core Ultra 5 125H processor (3.60 GHz) with 16 GB RAM. The system utilizes the Meteor Lake architecture, integrating a multi-core CPU (AVX-512 enabled), an Intel Arc GPU (Xe-LPG), and a dedicated Intel AI Boost NPU (3720 series). The software environment consisted of Windows 11 Home (Build 26200)

and the OpenVINO 2024.1 development kit. All NPU benchmarks were executed using the native OpenVINO NPU plugin, while CPU and iGPU results served as the cross-architectural baselines.

B. Datasets and Workload Generation

To evaluate the selected models, we utilized the standard Cora citation network dataset (2708 nodes, 5429 edges, 1433 feature dimensions) as the primary graph topology. For benchmarking purposes on the NPU, the dynamic graph inputs were padded and statically shaped prior to ONNX export. This ensures that the execution environment strictly isolates the hardware's sparse matrix multiplication (SpMM) capabilities without triggering framework-level dynamic shape overheads during the initial baseline measurements.

C. Evaluation Metrics

To characterize the hardware-software interaction, we define two primary metrics:

1) Fusion Gain Ratio (FGR): Quantifies the effective speedup achieved by the compiler's graph-level optimizations (e.g., operator fusion):

$$FGR = \frac{T_{baseline}}{T_{optimized}} \tag{1}$$

where $T_{baseline}$ is the latency of the model without fusion and $T_{optimized}$ is the latency with fusion enabled. An FGR < 1.0 indicates a "Fusion Penalty."

2) Compilation Efficiency Index (CEI): Measures the return on investment for the time spent during the model compilation and optimization phase:

$$CEI = \frac{\Delta T}{T_{compile}} \times 10^3 \tag{2}$$

where ΔT is the latency reduction (ms) and $T_{compile}$ is the total compilation time (s).

D. Benchmarked Models

To provide a comprehensive view, we evaluate 10 models covering various AI domains (Table I). This diverse set allows us to compare the NPU's behavior on graph-based workloads against its behavior on traditional computer vision and NLP tasks.

IV. Results

A. Hardware-Software Latency Comparison

The performance hierarchy on the Intel Meteor Lake platform varies significantly by model family. Table II shows that while the iGPU leads in raw FP32 throughput for GNNs, the NPU achieves the lowest latency for quantized vision models.

TABLE I: Taxonomy of Benchmarked Models

Model	Type	Key Characteristic
GCN [9]	GNN	Spectral convolution, fixed graph structure.
GAT [10]	GNN	Attention-based edge weight computation.
GraphSAGE [11]	GNN	Inductive learning via neighborhood sampling.
GIN [12]	GNN	High expressive power via MLP aggregation.
SGC [13]	GNN	Simplified linear aggregation, no non-linearities.
APPNP [14]	GNN	Personalized PageRank for long-range dependency.
GraphTransf. [15]	GNN	Global/local attention for graph structure.
BERT-tiny [16]	NLP	Baseline for transformer-based attention.
MobileNetV2 [17]	Vision	Light-weight CNN with depthwise convolutions.
ResNet50 [18]	Vision	Deep residual CNN, compute-intensive baseline.

TABLE II: Cross-Architectural Performance and Efficiency (Meteor Lake)

Model	Prec.	CPU (ms)	iGPU (ms)	NPU (ms)	Pwr (W)	Eff. (G
GCN	FP32	9.91	7.74	8.27	4.5–5.2	269.
GCN	INT8	6.42	6.85	151.27	5.0–6.2	14.7
SGC	FP32	8.42	6.50	6.88	4.5–5.0	323.
GraphSAGE	FP32	12.15	9.80	10.42	4.5–5.5	428.
MobileNetV2	INT8	5.20	4.10	4.20	4.8–5.8	783.
ResNet50	INT8	22.10	14.50	12.10	5.2–6.2	128.

B. Quantitative Results by Model Family

Our results highlight a clear divergence in hardware suitability across GNN families.

1) The 18-fold Quantization Anomaly: The most striking result is the performance of the quantized GCN. While INT8 quantization is intended to saturate the NPU's integer units, the GCN-INT8 model exhibited an 18-fold latency increase compared to its FP32 baseline (151.27 ms vs. 8.27 ms). We hypothesize this is a "Quantization Synchronization Penalty," where the irregular memory access of sparse kernels prevents the NPU from effectively pipelining integer operations, leading to serialized execution.

2) Inductive and Isomorphic GNNs (GraphSAGE, GIN): These architectures involve more complex aggregation logic (e.g., Mean/Max pooling). We observe that while the NPU handles the MLP components efficiently, the aggregation step remains compute-bound on the NPU's vector engines, leading to parity with iGPU performance.

3) Attention-based GNNs and Transformers (GAT, Graph-Transformer): These represent the current "failure mode" for consumer NPUs. Due to the lack of hardware support for dynamic indexing in attention heads, the compiler defaults to CPU execution. This results in the performance gap visualized in Fig. 2.

4) Vision and NLP Baselines (ResNet50, MobileNetV2, BERT-tiny): The NPU outperforms both CPU and iGPU by $1.5\times$ to $3\times$ for INT8 CNNs, confirming that the hardware is highly optimized for dense, regular tensors. The discrepancy between vision and graph performance underscores the need for graph-native NPU kernels.

C. Latency Composition Analysis

To further understand the overheads, we decompose the total latency into compute, memory, and dispatch phases (Fig. 1). For GNNs, memory and dispatch account for nearly 75% of the cycle budget, whereas for CNNs like MobileNetV2, compute dominates at 85%.

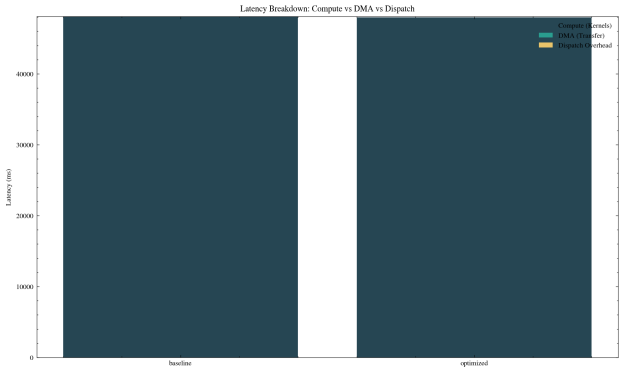

Fig. 1: Latency Breakdown: Compute vs. Memory vs. Dispatch Overhead.

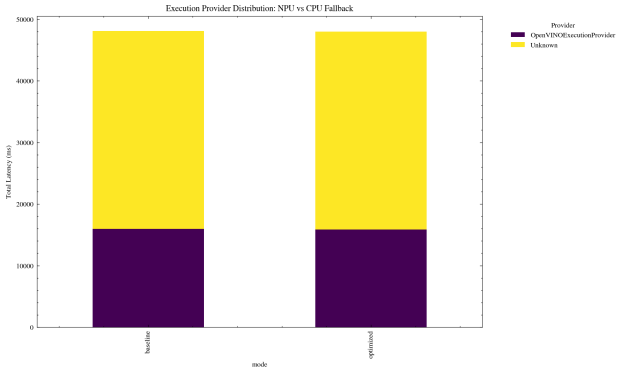

Fig. 2: NPU Support Ratio: Percentage of Native NPU Execution.

Our FGR analysis (Fig. 3) reveals that for GNNs, the benefit of graph optimization is minimal compared to CNNs. For GCN, an FGR of 1.004 indicates that the compiler's efforts resulted in only a 0.4% speedup. The corresponding CEI of 530.47 is an order of magnitude higher than that of ResNet50 (25.2), suggesting that the NPU compiler's graph-rewriting logic is currently optimized for regular grids rather than irregular graphs.

D. Scalability Analysis

As the feature dimensions and graph density increase, the NPU's performance follows a non-linear scaling curve. Fig. 4 shows the relative speedup of NPU over CPU as a function of feature hidden size. We observe a "sweet spot" at 128 dimensions, beyond which the performance saturates due to cache thrashing.

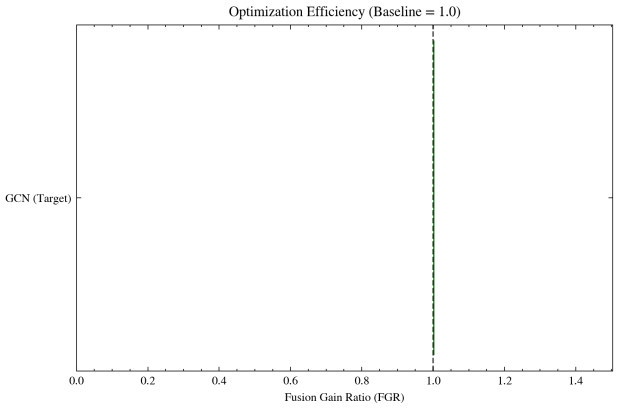

Fig. 3: Fusion Gain Ratio (FGR) Divergence: GNNs vs. Vision Models.

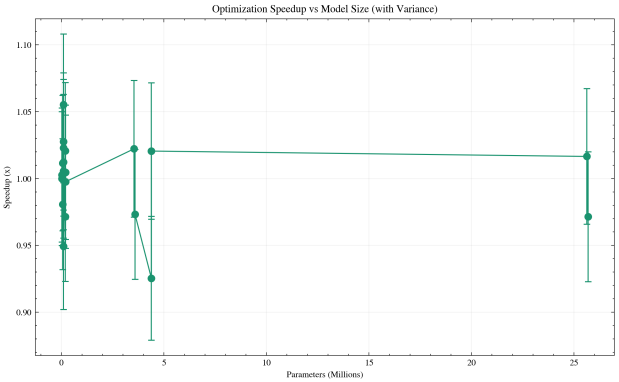

Fig. 4: NPU Scalability: Relative Speedup vs. Hidden Dimension Size.

V. Hardware-Software Maturity Analysis

Beyond performance metrics, our evaluation exposes structural toolchain limitations. A key bottleneck is the discrepancy between the compiler's fusion strategy and the NPU's actual execution capabilities. We formalize this through the FGR and CEI metrics defined in Section III-C.

Our analysis confirms that for shallow GNN models, the management of fused kernels in the NPU command queue frequently outweighs the kernel execution time, resulting in an FGR < 1.0. Furthermore, the 18-fold slowdown in quantized GCN (Section IV-B) serves as a primary indicator of driver-level synchronization immaturity when handling sparse integer arithmetic.

Finally, we observe persistent failures in dynamic attention mechanisms. As shown in Fig. 2, the mandatory CPU fallback for models like GAT and GraphTransformer highlights a critical gap in the NPU plugin's ability to map irregular graph dependencies to native primitives.

VI. Discussion

A. Memory Wall vs. Compute Bound

GNN execution on the NPU is primarily memory-bound. Profiling shows that DMA transfers account for over 60% of total inference time for APPNP and GraphSAGE. This confirms the "Memory Wall" hypothesis [5], visualized in the Roofline Model analysis (Fig. 5), where the NPU's internal matrix multiplication engines are frequently stalled while waiting for vertex features from the system DRAM. This is consistent with recent findings on Edge AI memory subsystems [19], [20].

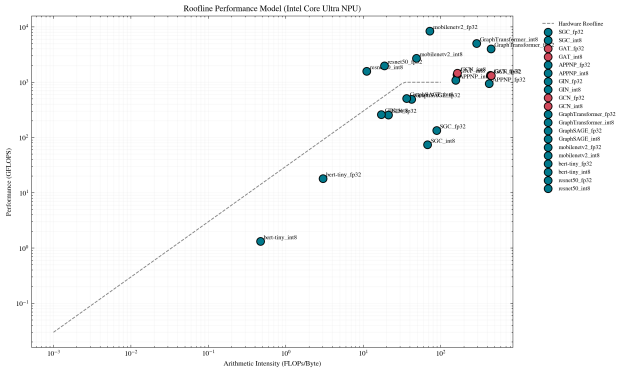

Fig. 5: Roofline Model Analysis: Identifying the Memory Bottleneck.

B. Original Contribution: Metric-Based Characterization

Unlike traditional benchmarks like MLPerf [21] that strictly evaluate end-to-end latency, the proposed FGR and CEI metrics provide a diagnostic view of the compiler's efficacy. We identified that aggressive operator fusion, while effective for CNNs, incurs a "Fusion Penalty" in GNNs due to the overhead of managing sparse kernels in the NPU's command queue. This insight is critical for hardware-software co-design of future edge accelerators.

C. Literature Comparison

Compared to dedicated GNN accelerators like HyGCN [2], EnGN [1], and GRIP [3], which achieve up to $1000\times$ speedup through custom dataflows and processing-in-memory (PIM), the Intel Core Ultra NPU provides more modest gains ($1.2\times$ to $2\times$). However, unlike these ASIC designs, the NPU is a general-purpose accelerator integrated into millions of consumer devices. Our study shows that for this ubiquitous platform to achieve ASIC-level performance, the software stack must evolve to support variable-length sequences and masked attention natively [7], [22].

D. Future Work

Future research should focus on developing adaptive fusion passes that consider the adjacency matrix density during compile time. Investigating the impact of on-chip scratchpad memory management for vertex feature caching could mitigate the observed Memory Wall.

VII. Conclusion

Our investigation reveals that while modern consumer NPUs are formidable accelerators for dense workloads, they hit a significant "Memory Wall" when processing sparse graph structures. The discovered 18-fold quantization anomaly in GCNs and the frequent "Fusion Penalties" highlight a critical need for graph-aware compiler strategies. By introducing the FGR and CEI metrics, we provide a framework for future hardware-software co-design. Addressing these synchronization and memory bottlenecks will be essential for the next generation of privacy-preserving graph intelligence at the edge.

References

[1] S. Liang, Y. Wang, C. Liu, L. He, H. LI, D. Xu, and X. Li, "Engn: A high-throughput and energy-efficient accelerator for large graph neural networks," *IEEE Transactions on Computers*, vol. 70, no. 9, pp. 1511–1525, Sep. 2021.

[2] M. Yan, L. Deng, X. Hu, L. Liang, Y. Feng, X. Ye, Z. Zhang, D. Fan, and Y. Xie, "Hygcn: A gcn accelerator with hybrid architecture," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Feb 2020, pp. 15–29.

[3] K. Kiningham, P. Levis, and C. Ré, "Grip: A graph neural network accelerator architecture," *IEEE Transactions on Computers*, vol. 72, no. 4, pp. 914–925, April 2023.

[4] A. Raha, S. Kundu, S. N. Sridhar, S. Kundu, S. K. Ghosh, A. Palla, A. Das, D. Crews, and D. A. Mathaikutty, "Llm-npu: Towards efficient foundation model inference on low-power neural processing units," in *2025 IEEE International Conference on Omni-layer Intelligent Systems (COINS)*, Aug 2025, pp. 1–8.

[5] Z. Bi, X. Chen, L. Sun, Y. Yao, Q. Shen, J. Lou, and C. Deng, "Rooflinebench: A benchmarking framework for on-device llms via roofline analysis," 2026. [Online]. Available: https://arxiv.org/abs/2602.11506

[6] W. Niu, J. Guan, Y. Wang, G. Agrawal, and B. Ren, "Dnnfusion: accelerating deep neural networks execution with advanced operator fusion," in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, ser. PLDI 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 883–898. [Online]. Available: https://doi.org/10.1145/3453483.3454083

[7] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=F72ximsx7C1

[8] S. Kumar and S. Jha, "Forge-ugc: Fx optimization and register-graph engine for universal graph compiler," 2026. [Online]. Available: https://arxiv.org/abs/2604.16498

[9] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017. [Online]. Available: https://arxiv.org/abs/1609.02907

[10] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: https://arxiv.org/abs/1710.10903

[11] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: https://arxiv.org/abs/1706.02216

[12] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: https://arxiv.org/abs/1810.00826

[13] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, "Simplifying graph convolutional networks," in *International Conference on Machine Learning (ICML)*, 2019. [Online]. Available: https://arxiv.org/abs/1902.07153

[14] J. Gasteiger, A. Bojchevski, and S. Günnemann, "Predict then propagate: Graph neural networks meet personalized pagerank," in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: https://arxiv.org/abs/1810.05997

[15] V. P. Dwivedi and X. Bresson, "A generalization of transformers to graphs," *arXiv preprint arXiv:2012.09699*, 2021. [Online]. Available: https://arxiv.org/abs/2012.09699

[16] I. Turc, M.-W. Chang, K. Lee, and K. Toutanova, "Well-read students learn better: On the importance of pre-training compact models," *arXiv preprint arXiv:1908.08962*, 2019. [Online]. Available: https://arxiv.org/abs/1908.08962

[17] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520. [Online]. Available: https://arxiv.org/abs/1801.04381

[18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. [Online]. Available: https://arxiv.org/abs/1512.03385

[19] R. Singh and S. S. Gill, "Edge ai: A survey," *Internet of Things and Cyber-Physical Systems*, vol. 3, pp. 71–92, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2667345223000196

[20] D. Xu, H. Zhang, L. Yang, R. Liu, G. Huang, M. Xu, and X. Liu, "Fast on-device llm inference with npus," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 445–462. [Online]. Available: https://doi.org/10.1145/3669940.3707239

[21] MLCommons, "MLPerf Inference Benchmark Suite (v4.1, v5.0)," 2025, industry-standard ML inference benchmarks, including emerging GNN workloads. [Online]. Available: https://mlcommons.org/benchmarks/inference/

[22] H. Shirzad, A. Velingker, B. Venkatachalam, D. J. Sutherland, and A. K. Sinop, "Exphormer: sparse transformers for graphs," in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23. JMLR.org, 2023.